



Use Case Diagram

แนวคิด องค์ประกอบ และการประยุกต์ใช้ใน Software Development

Use Case Diagram เป็นแผนภาพประเภทหนึ่งของ UML (Unified Modeling Language) ที่ใช้แสดงความสัมพันธ์ระหว่างผู้ที่เกี่ยวข้องกับระบบและความสามารถที่ระบบต้องให้บริการ โดยมุ่งตอบคำถามว่า "ใครต้องการทำอะไรกับระบบ"

Use Case Diagram มีบทบาทสำคัญใน Requirements Engineering เพราะช่วยให้นักวิเคราะห์ นักพัฒนา ผู้ทดสอบ และผู้มีส่วนได้ส่วนเสียมองเห็นขอบเขตและความสามารถหลักของระบบในภาพรวมเดียวกัน โดยอธิบาย พฤติกรรมของระบบจากมุมมองของผู้ใช้ ไม่ใช่โครงสร้างภายในหรือขั้นตอนการเขียนโปรแกรม



Introduction to Use Case Modeling

Use Case คืออะไร

Use Case คือคำอธิบายพฤติกรรมหรือบริการที่ระบบมอบให้แก่ Actor เพื่อให้ Actor บรรลุเป้าหมายบางอย่าง สิ่งสำคัญคือ Use Case ควรสื่อถึง **Goal ของ Actor** มากกว่าองค์ประกอบของโปรแกรม

✓ เหมาะสม

- Register Account
- Login
- Search Product
- Place Order
- Make Payment
- Track Order

✗ ไม่เหมาะสม

ชื่ออย่าง "Order Button" ไม่เหมาะสม เพราะผู้ใช้ไม่ได้มีเป้าหมายว่า "ต้องการกดปุ่ม" แต่ต้องการ "สั่งซื้อสินค้า"

Use Case ควรสะท้อน Goal ที่มีคุณค่าต่อ Actor เสมอ

Use Case Diagram และจุดประสงค์

Use Case Diagram แสดงความสัมพันธ์ระหว่าง Actor → System → Use Case ช่วยให้เห็นภาพรวมของ Functional Requirements โดยไม่ลงรายละเอียดการประมวลผลภายใน

1

แสดงความสามารถหลัก

ระบุ Functional Requirements ของระบบในภาพรวม

2

ระบุผู้เกี่ยวข้อง

กำหนด Actor และขอบเขตของระบบอย่างชัดเจน

3

สื่อกลาง

เชื่อมผู้ใช้กับทีมพัฒนา และใช้เป็นจุดเริ่มต้นของ Test Scenario

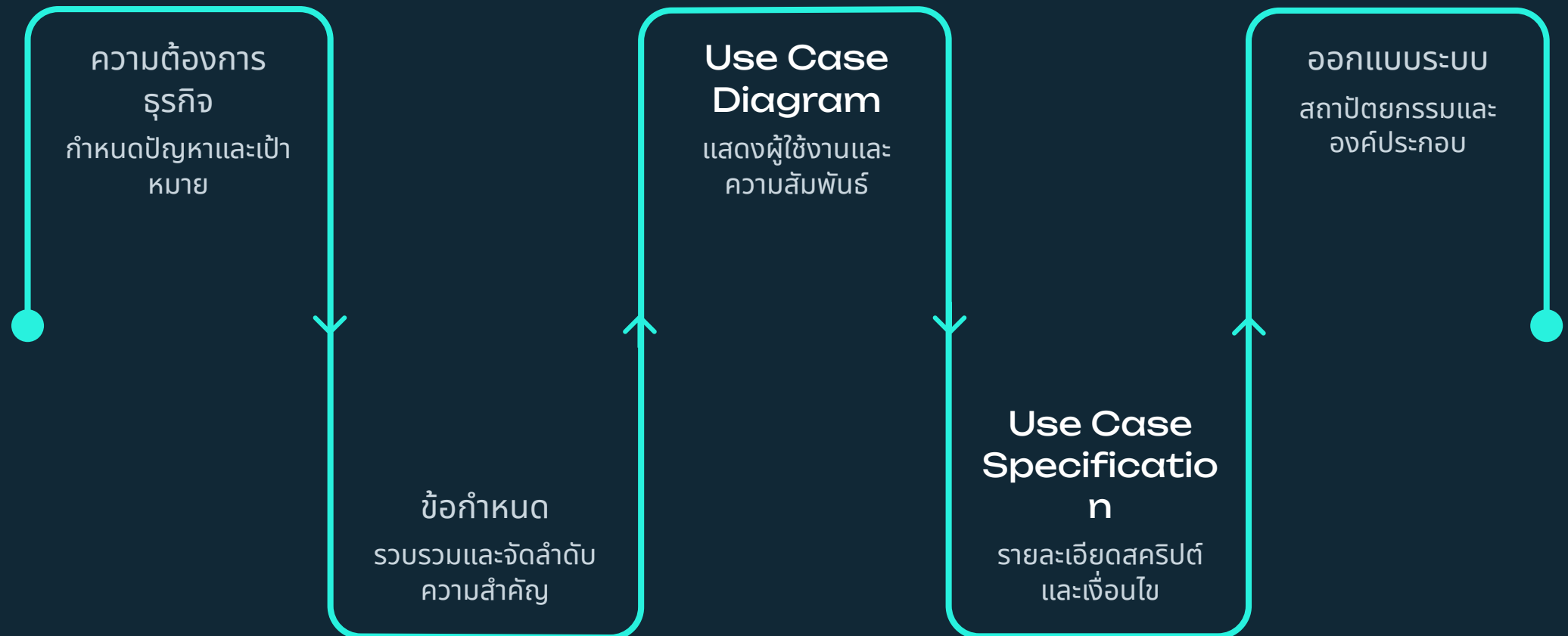
4

Traceability

เชื่อมโยง Requirement ID, User Story, Design Component และ Test Case

Use Case Diagram ใน Software Development Life Cycle

Use Case Diagram มักถูกสร้างในช่วง Requirements Analysis แต่ใช้อ้างอิงได้ตลอด SDLC — ไม่ใช่เพียงแผนภาพนำเสนอ Requirement แต่เป็นตัวกลางเชื่อม Requirement กับ Design, Implementation และ Testing



ข้อดีและข้อจำกัดของ Use Case Diagram

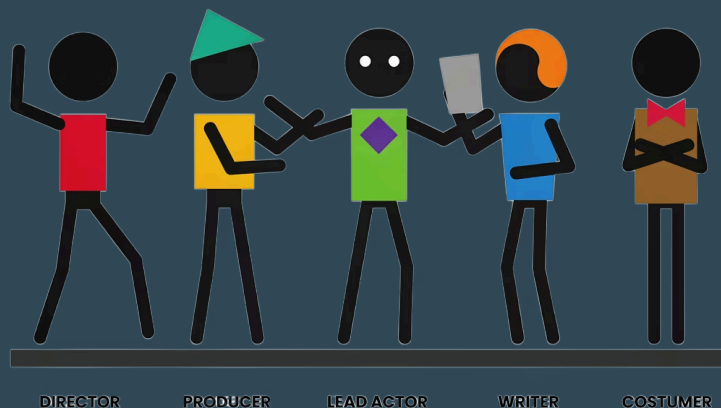
ข้อดี

- เข้าใจง่าย แม้ไม่มีพื้นฐานเขียนโปรแกรม
- กำหนดขอบเขตระบบได้ชัดเจน
- ตรวจสอบความครบถ้วนของ Requirements
- สร้าง Traceability กับ Requirement ID และ Test Case

ข้อจำกัด

ไม่สามารถอธิบายรายละเอียดได้ครบ เช่น Algorithm, Data Structure, Database Schema, UI, Business Logic, Performance และ Security Requirement

จึงควรใช้ร่วมกับ Use Case Specification, Activity Diagram, Sequence Diagram และ Non-functional Requirements



Elements of Use Case Diagram

Actor

Actor คือบทบาทของบุคคล องค์กร อุปกรณ์ หรือระบบภายนอกที่มีปฏิสัมพันธ์กับระบบ Actor ไม่จำเป็นต้องเป็นมนุษย์เสมอไป

Human Actor

Customer, Administrator, Employee, Instructor, Student

System Actor

Payment Gateway, Email Service, Delivery Service, External Accounting System

Primary Actor vs Supporting Actor และ Use Case

Primary Actor

Actor ที่เริ่มต้นการทำงานของ Use Case เพื่อบรรลุเป้าหมายของตนเอง

ตัวอย่าง: **Customer** → **Place Order**

Supporting Actor

Actor ที่ระบบเรียกใช้เพื่อช่วยให้ Use Case สำเร็จ

ตัวอย่าง: **Payment Gateway** ใน **Make Payment**

Use Case & System Boundary

Use Case แสดงด้วย วงรี และชื่อควรเป็นคำกริยา เช่น Place Order, Generate Report

System Boundary คือกรอบที่ระบุว่า Use Case ไດอยู่ภายในระบบ ส่วน Actor จะอยู่ภายนอก System Boundary เสมอ

Association และ Generalization

Association

ความสัมพันธ์ระหว่าง Actor กับ Use Case แสดงด้วยเส้นตรง ไม่ได้หมายความว่า Actor เป็นเจ้าของ Use Case แต่มีส่วนร่วมกับ Use Case นั้น

Customer — Place Order

Generalization (Actor)

ใช้แสดงความสัมพันธ์แบบสืบทอดระหว่าง Actor เช่น User เป็น Parent ของ Customer และ Admin ทำให้ทั้งคู่สามารถ Login ได้โดยไม่ต้องเชื่อมต่อซ้ำ

«include» และ «extend»

«include»

Use Case หนึ่งจำเป็นต้องเรียกใช้อีก Use Case เสมอ เหมาะสำหรับแยกพฤติกรรมที่ถูกใช้ซ้ำจากหลาย Use Case

ตัวอย่าง: Withdraw Money และ Transfer Money ต่างก็ «include» Authenticate Customer

«extend»

พฤติกรรมเพิ่มเติมที่เกิดขึ้นเฉพาะบางเงื่อนไข Base Use Case ทำงานได้สมบูรณ์โดยไม่มี Extension

ตัวอย่าง: Apply Discount «extend» Place Order — เกิดเฉพาะเมื่อผู้ใช้มี Coupon

ตัวอย่าง: Request OTP «extend» Login — เกิดเฉพาะ Login จากอุปกรณ์ใหม่

Identifying Actors

การวิเคราะห์ Actor ควรเริ่มจากการตั้งคำถาม เช่น ใครเป็นผู้ใช้ระบบ? ใครได้รับผลลัพธ์? ระบบภายนอกใดต้องแลกเปลี่ยนข้อมูล?



Actor = Role ไม่ใช่บุคคล

นายสมชายอาจเป็นทั้ง Instructor, Program Coordinator และ Administrator — ถือเป็น 3 Actor แม้เป็นคนเดียวกัน



ระบบภายนอกเป็น Actor ได้

Payment Gateway, Bank System, Email Service เป็น Actor ได้ แต่ Database ที่อยู่ภายในระบบเดียวกันไม่ควรเป็น Actor



ตั้งชื่อตามบทบาท

ใช้ชื่อเช่น Customer, Administrator, Instructor — หลีกเลี่ยง Somchai, User 1, Person, Computer

ข้อผิดพลาดในการระบุ Actor

ข้อผิดพลาดสำคัญคือการนำทุกสิ่งที่เกี่ยวข้องกับระบบมาเป็น Actor สิ่งต่อไปนี้เป็น Actor ก็ต่อเมื่ออยู่นอก **System Boundary** และมีปฏิสัมพันธ์กับระบบที่กำลังศึกษาเท่านั้น

ไม่ควรเป็น Actor

Database, Web Server, Mobile Application, Controller, API — สิ่งเหล่านี้เป็น Component ภายในระบบ

ควรเป็น Actor

Payment Gateway, Delivery Service, Email Service — เพราะอยู่นอก System Boundary และมีปฏิสัมพันธ์กับระบบ

Identifying Use Cases

เริ่มจาก **Goal** ของ **Actor**

ถามว่า "Actor ต้องการบรรลุเป้าหมายอะไรเมื่อใช้ระบบ?"

ตั้งชื่อด้วยรูปแบบ **Verb + Object** เช่น Create Account, Place Order, Generate Report

หลีกเลี่ยงชื่อที่เป็นคำนามอย่างเดียว เช่น Order, Product, Report เพราะไม่สื่อว่าระบบต้องทำอะไร

ระดับความละเอียดที่เหมาะสม

เล็กเกินไป: Click Button, Input Name, Validate Email

ใหญ่เกินไป: Manage System, Operate Business

เหมาะสม: Register Customer, Place Order, Approve Loan, Generate Monthly Report

User Goal vs System Function

นักศึกษาบางคนแยกไม่ออกระหว่าง Goal กับฟังก์ชันระดับต่ำ และ Business Process อาจกว้างกว่า Use Case ของ Software

1

Business Process

ขายสินค้าให้ลูกค้า (กว้างเกินไปสำหรับ Use Case เดียว)

2

Software Use Cases

Search Product, Create Order, Make Payment, Issue Receipt, Arrange Delivery

❏ Use Case หนึ่งควรเป็นกิจกรรมที่มีคุณค่าหรือผลลัพธ์ที่มีความหมายต่อ Actor — ไม่ใช่ UI Interaction เช่น Enter Username หรือ Click Login Button

Relationships in Use Case Diagram

1 Association

เชื่อม Actor กับ Use Case ด้วยเส้นตรง
แสดงว่า Actor มีปฏิสัมพันธ์กับ Use Case นั้น

2 Actor Generalization

ใช้เมื่อ Actor หลายประเภทมีพฤติกรรมร่วมกัน เช่น User เป็น Parent ของ Customer และ Staff

3 Use Case Generalization

ใช้เมื่อ Use Case หนึ่งเป็นรูปแบบเฉพาะของอีก Use Case เช่น Pay by Card และ Pay by QR เป็นรูปแบบเฉพาะของ Make Payment ควรใช้เท่าที่จำเป็น

เปรียบเทียบ «include» กับ «extend»

ประเด็น	«include»	«extend»
ลักษณะ	พฤติกรรมที่ถูกเรียกใช้เป็นส่วนหนึ่งของ Use Case	พฤติกรรมเพิ่มเติม
การเกิด	เกิดตามโครงสร้างของ Base Use Case เสมอ	เกิดเมื่อมีเงื่อนไขเฉพาะ
วัตถุประสงค์	Reuse พฤติกรรมร่วม	รองรับ Variation
Base Use Case	ต้องอาศัย Included Behavior	สมบูรณ์ได้โดยไม่มี Extension

❏ ไม่ควรจำเพียงว่า include = ต้องทำ / extend = ไม่ต้องทำ แต่ควรเข้าใจเจตนาของความสัมพันธ์ด้วย

System Boundary, Scope และ Scope Creep

System Boundary

กำหนดว่าจะอะไรเป็นความรับผิดชอบของระบบ Actor และ Use Case ขึ้นอยู่กับ System Boundary

ตัวอย่าง: หากทีมพัฒนา Payment Gateway เอง องค์ประกอบภายในจะไม่ถือเป็น Actor อีกต่อไป

Scope Creep

Use Case Diagram ช่วยควบคุม Scope ได้ เมื่อมี Requirement ใหม่ เช่น "ระบบต้องแนะนำสินค้าด้วย AI" สามารถถามได้ว่า Recommendation System อยู่ใน Scope เดิมหรือไม่

การเพิ่ม Use Case ใหม่อาจนำมาซึ่ง Database, Algorithm, API, Testing และ Infrastructure เพิ่มเติม ดังนั้น Use Case Diagram จึงใช้เป็นจุดเริ่มต้นของ Impact Analysis ได้

จาก Requirements สู่ Use Case Diagram

พิจารณา Requirement: "ระบบร้านค้าออนไลน์อนุญาตให้ลูกค้าสมัครสมาชิก เข้าสู่ระบบ ค้นหาสินค้า เพิ่มสินค้าในตะกร้า สั่งซื้อ และชำระเงินออนไลน์ ระบบเชื่อมต่อ Payment Gateway และบริษัทขนส่ง"



ค้นหาระบบ

ค้นหาตัวละคร

ค้นหาเป้าหมายผู้
ใช้

ค้นหาปฏิสัมพันธ์
ภายนอก

สร้าง Use
Case



เทคนิคการหา Use Case จาก ข้อความ Requirement

ให้มองหาคำกริยา เช่น สมัคร, ค้นหา, เพิ่ม, แก้ไข, ยกเลิก, ตรวจสอบ, อนุมัติ, สั่งซื้อ, ชำระเงิน, ออก
รายงาน

- ❏ ไม่ควรเปลี่ยนคำกริยาทุกคำเป็น Use Case กันทีเดียว ต้องตรวจสอบก่อนว่าคำนั้นแสดง Goal ที่มีคุณค่าต่อ Actor หรือไม่

Use Case Specification

Use Case Diagram แสดงเพียงภาพรวม หากต้องการนำ Requirement ไปพัฒนา Software จำเป็นต้องมี **Use Case Specification** ที่มีรายละเอียดเพิ่มเติม

Use Case ID

UC-001, UC-002 เพื่อสร้าง
Traceability: REQ-12 → UC-003 → TC-025

Goal & Actors

ระบุวัตถุประสงค์ของ Actor,
Primary Actor และ Supporting Actor

Trigger

เหตุการณ์ที่ทำให้ Use Case เริ่ม
ต้น เช่น ลูกค้าเลือกปุ่ม
"Checkout"

Pre/Postconditions

เงื่อนไขก่อนเริ่ม และสถานะที่ต้อง
เกิดขึ้นหลัง Use Case สิ้นสุด

Business Rules

กฎทางธุรกิจที่ Use Case ต้องปฏิบัติตาม เช่น BR-01 ยอดสั่งซื้อขั้นต่ำ 300 บาท

Special Requirements

Performance, Security, Availability เช่น ระบบต้องพร้อมให้บริการ 99.9%

Main Flow, Alternative Flow และ Exception Flow

Main Flow (Happy Path)

เส้นทางหลักเมื่อทุกอย่างทำงานตามปกติ เช่น UC-003 Place Order: Customer เลือก Checkout → System แสดงรายการ → ระบุที่อยู่ → คำนวณยอด → ยืนยัน → สร้าง Order → แสดงหมายเลข Order

Alternative Flow

เส้นทางที่แตกต่างจาก Main Flow แต่ยังสามารถสำเร็จได้ เช่น 3a. ลูกค้าเลือกที่อยู่เดิมที่บ้านที่กไว้ หรือ 4a. ลูกค้าใช้ Coupon ส่วนลด

Exception Flow

กรณีผิดพลาด เช่น 4E1. สินค้าบางรายการหมด ระบบแจ้งลูกค้าและไม่อนุญาตให้ยืนยัน จนกว่าจะปรับรายการสินค้า

📌 เริ่มเขียน Happy Path ก่อน แล้วถามว่า "แต่ละ Step สามารถเกิดอะไรผิดปกติได้บ้าง?" วิธีนี้ช่วยค้นหา Alternative และ Exception Flow ได้ง่าย

ตัวอย่าง Use Case Login: Flow ครบทั้ง 3 ประเภท

Main Flow

1. User เลือก Login
2. System แสดงแบบฟอร์ม
3. User กรอก Email และ Password
4. System ตรวจสอบข้อมูล
5. System ยืนยันตัวตนสำเร็จ
6. System แสดงหน้า Dashboard

Alternative Flow

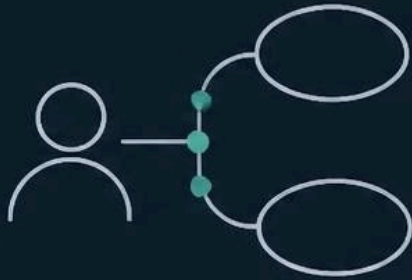
3a. User เลือก Remember Me → System บันทึก Authentication Token → กลับไป Step 4

Exception Flow

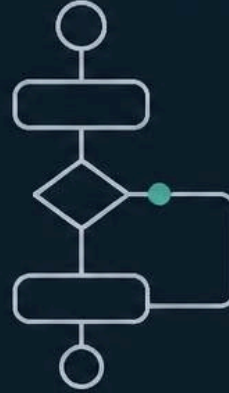
4E1. Password ไม่ถูกต้อง → System แสดงข้อความผิดพลาด → ให้ User กรอกใหม่

4E2. Account ถูกระงับ → System ปฏิเสธการ Login → แสดงช่องทางติดต่อ Administrator

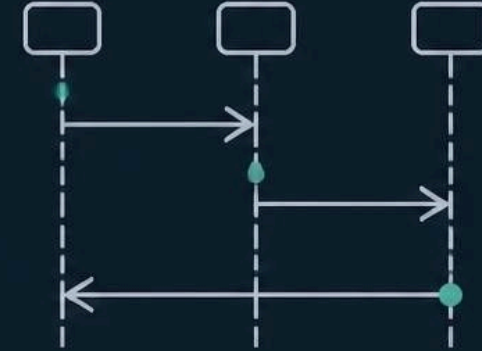
Use Case Diagram vs Other UML Diagrams



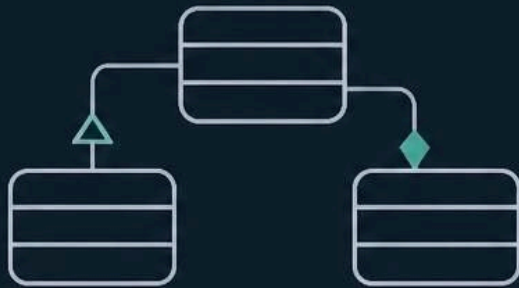
แผนภาพกรณีใช้งาน
(ใครใช้ระบบและต้องการอะไร?)



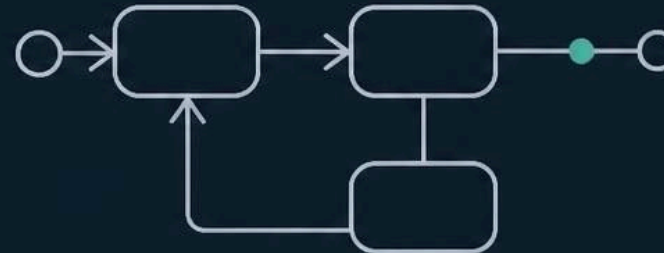
แผนภาพกิจกรรม
(เวิร์กโฟลว์ไหลอย่างไร?)



แผนภาพลำดับ
(ออบเจกต์สื่อสารกันอย่างไรตามเวลา?)



แผนภาพคลาส
(มีคลาสอะไรบ้างและเกี่ยวข้องกันอย่างไร?)



แผนภาพสถานะ
(ออบเจกต์เปลี่ยนสถานะอย่างไร?)

ความสัมพันธ์ระหว่าง UML Diagrams

ไม่จำเป็นต้องสร้างทุก Diagram สำหรับทุก Use Case แต่ควรเลือก Diagram ให้เหมาะสมกับสิ่งที่ต้องการวิเคราะห์



Use Case และ Software Testing

Use Case เป็นแหล่งข้อมูลสำคัญในการสร้าง Software Test ตามหลักการ Use Case → Scenario → Test Case



Positive Testing

Main Flow → Test Case ที่ Email และ Password ถูกต้อง → Expected: Login สำเร็จ



Alternative Flow Testing

User Login โดยเลือก Remember Me → ทดสอบว่า Token หรือ Session ถูกสร้างตาม Requirement



Negative Testing

Exception Flow → Password ผิด, Account ถูก Lock, Email ไม่มีในระบบ, Server ไม่ตอบสนอง

Requirement Traceability

Use Case สามารถเป็นตัวกลางเชื่อม Requirement กับ Test Case ทำให้ตรวจสอบได้ว่า Requirement ทุกข้อได้รับการพัฒนาและทดสอบแล้วหรือไม่

Requirement	Use Case	Test Case
REQ-01	UC-01 Register Account	TC-001 ถึง TC-005
REQ-02	UC-02 Login	TC-006 ถึง TC-012
REQ-03	UC-03 Place Order	TC-013 ถึง TC-025
REQ-04	UC-04 Make Payment	TC-026 ถึง TC-040

Common Mistakes in Use Case Diagram

1

Database เป็น Actor

หาก Database เป็นส่วนหนึ่งของระบบ ไม่ควรเป็น Actor ระบบควรจัดการ Database ภายในเอง

2

Actor อยู่ใน System Boundary

Actor ต้องอยู่นอก System Boundary เสมอ

3

ชื่อหน้าจอเป็น Use Case

Login Page → Login, Registration Page → Register Account

4

Use Case เล็ก/ใหญ่เกินไป

Click Button เล็กเกินไป, Manage University ใหญ่เกินไป

5

Include/Extend มากเกินไป

Diagram ที่มี Relationship ซับซ้อนจนอ่านไม่รู้เรื่อง ขัดกับจุดประสงค์ของ Use Case Diagram

6

แสดง Sequence ใน Diagram

ไม่ควรใช้ลูกศร Login → Search → Cart → Order เพื่อแสดงลำดับ ควรใช้ Activity Diagram แทน

Case Study: ระบบจองห้องพักโรงแรม

Actors

- Customer
- Hotel Staff
- Payment Gateway
- Email Service

Use Cases ของ Customer

- Register Account / Login
- Search Available Room
- Book Room / Make Payment
- View Booking / Cancel Booking

Use Cases ของ Hotel Staff

- Manage Room / Manage Room Rate
- View & Update Booking Status
- Generate Booking Report

Include & Extend

«include»: Book Room → Check Room Availability, Calculate Booking Price

«extend»: Apply Promotion «extend» Book Room — เพราะลูกค้าอาจมีหรือไม่ใช้ Promotion

ตัวอย่าง Use Case Specification:

UC-05 Book Room

หัวข้อ	รายละเอียด
Goal	ลูกค้าต้องการจองห้องพักในช่วงวันที่ที่ต้องการ
Primary Actor	Customer
Supporting Actors	Payment Gateway, Email Service
Trigger	ลูกค้าเลือกห้องพักและกด Book Now
Preconditions	ลูกค้าเข้าสู่ระบบแล้ว / มีห้องว่างในช่วงวันที่เลือก
Postconditions	Booking ถูกสร้าง / ระบบสร้าง Booking Number
Business Rules	BR-01 อายุ ≥ 18 ปี / BR-02 ใช้ Promotion Code ได้ 1 Code/Booking / BR-03 ยกเลิก >3 วันคืนเงินเต็มจำนวน
Exception E1	ห้องถูกจองโดยลูกค้าคนอื่น → ยกเลิก Booking → แจ้งลูกค้า → แนะนำห้องอื่น

จาก Use Case สู่ Design และ Code

Use Case → Sequence Diagram

จาก Book Room สามารถวิเคราะห์ Component ที่ต้องมี เช่น Customer → Booking UI → BookingController → RoomService → BookingService → Database ช่วยให้เห็นว่า Use Case เป็น Input สำคัญของ Software Design

Use Case → Class Diagram

จาก Book Room ค้นพบ Domain Object เช่น Customer, Room, RoomType, Booking, Payment, Promotion และความสัมพันธ์ Customer makes Booking reserves Room

Use Case → Source Code

Use Case ไม่ควรถูกแปลงเป็น Class แบบ 1:1 เสมอไป Place Order อาจเกี่ยวข้องกับ OrderController, OrderService, InventoryService, PaymentService, OrderRepository, NotificationService — ต้องแยกให้ออกระหว่าง Functional Requirement กับ Software Design

Use Case กับ Agile และ Non-functional Requirements

Use Case กับ Agile

ใน Agile อาจไม่จัดทำ Use Case Specification ขนาดใหญ่ แต่แนวคิดยังมีประโยชน์ โดยเชื่อมโยงกับ Epic, Feature, User Story และ Acceptance Criteria

ตัวอย่าง: Use Case "Track Order" → User Story: "As a customer, I want to track my order so that I know when my package will arrive."

Use Case และ User Story ไม่ได้เป็นคู่แข่งกัน แต่ใช้ร่วมกันได้

Use Case กับ Non-functional Requirements

Use Case เหมาะสำหรับ Functional Requirements เท่านั้น สิ่งต่อไปนี้ไม่ควรสร้างเป็น Use Case โดยตรง:

- ระบบต้องตอบสนองภายใน 2 วินาที
- รองรับผู้ใช้พร้อมกัน 10,000 คน
- Password ต้องถูกเข้ารหัส

แต่สามารถผูก NFR เข้ากับ Use Case ที่เกี่ยวข้องได้ เช่น UC-02 Login + NFR-PERF-02

แนวทางตรวจสอบคุณภาพและกิจกรรมในชั้นเรียน

Checklist ตรวจสอบ Diagram

- Actor ทุกตัวอยู่นอก System Boundary?
- ชื่อ Use Case เป็น Verb + Object?
- Use Case แสดง Goal ของ Actor?
- Include ใช้กับพฤติกรรมร่วมที่เหมาะสม?
- Extend มีเงื่อนไขชัดเจน?
- ไม่มี Internal Component ถูกนำมาเป็น Actor?

กิจกรรมในชั้นเรียน

โจทย์: ระบบลงทะเบียนเรียนออนไลน์ของมหาวิทยาลัย ให้นักศึกษาดำเนินการ:

1. ระบุ System Boundary และ Actor
2. หา User Goal และ Use Case
3. สร้าง Use Case Diagram
4. วิเคราะห์ include/extend/generalization
5. เขียน Use Case Specification พร้อม Flow ครบทั้ง 3 ประเภท
6. สร้าง Test Scenario และ Sequence Diagram